

On-Policy Temporal-Difference Control: SARSA(0)

Reinforcement Learning

Sai Sampath Kedari

sampath@umich.edu

Contents

1	Setup	2
1.1	Policy Evaluation	2
1.2	Policy Improvement (ε -greedy)	2
1.3	The Full Algorithm	2
2	From Operator to Sample	3
2.1	The Statistical Move	3
2.2	The Two Forms of q_π	3
2.3	Rewriting the Bellman Equation as a Conditional Expectation	3
2.4	Replacing Each Expectation with One Sample	5
2.5	Running-Mean Update	7
3	SARSA(0): Online GPI	8
3.1	From Phased GPI to Online GPI	8
3.2	On-Policy Sampling	9
3.3	The Algorithm	9
3.4	Convergence	10

1 Setup

We work throughout with an infinite-horizon discounted MDP with finite state space $\mathcal{S}$, finite action sets $\mathcal{A}(s)$, joint transition kernel

$$p(s', r \mid s, a) = \Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a),$$

discount factor $\gamma \in [0, 1)$, and stationary policies $\pi(a \mid s)$.

The control problem is to find an optimal policy. Every algorithm in this report alternates two aspects: *policy evaluation* (given π , estimate q_π) and *policy improvement* (given q_π , construct a better policy). We state both in their model-based DP form here as the reference point. Subsequent sections replace the model-based evaluation with sampled approximations to obtain SARSA(0), n -step SARSA, and SARSA(λ); the improvement step stays ε -greedy throughout.

1.1 Policy Evaluation

Iterate the Bellman expectation operator on Q starting from any Q_0 :

$$Q_{k+1}(s, a) = r(s, a) + \gamma \sum_{s'} p(s' \mid s, a) \sum_{a'} \pi(a' \mid s') Q_k(s', a').$$

$Q_k \rightarrow q_\pi$ for any Q_0 (Report 10). In practice we stop after k iterations with $Q_k \approx q_\pi$.

1.2 Policy Improvement (ε -greedy)

Given the current estimate Q , define

$$\pi'(a \mid s) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}(s)|} & a = \arg \max_{a'} Q(s, a'), \\ \frac{\varepsilon}{|\mathcal{A}(s)|} & \text{otherwise.} \end{cases}$$

The ε -greedy Policy Improvement Theorem (Report 17) guarantees $v_{\pi'} \geq v_\pi$ within the class of ε -soft policies. Every action receives probability at least $\varepsilon/|\mathcal{A}(s)|$, which is what allows model-free sample-based evaluation in later sections to keep visiting all state-action pairs.

1.3 The Full Algorithm

Start from arbitrary Q_0 and arbitrary π_0 . Alternate:

$$Q_0 \xrightarrow{k \text{ steps of } T_{\pi_0}} Q_{\pi_0} \xrightarrow{\varepsilon\text{-greedy}} \pi_1 \xrightarrow{k \text{ steps of } T_{\pi_1}} Q_{\pi_1} \xrightarrow{\varepsilon\text{-greedy}} \pi_2 \longrightarrow \dots$$

Both steps require the model: evaluation needs p inside T_π , and the operator iteration runs k sweeps over the full state-action space before improvement. This is the reference algorithm. The next section converts the evaluation step from full expectation to one sample, giving SARSA. Section 3 weave the ε -greedy improvement into the sample-based evaluation, updating π after every single sample rather than between phases.

2 From Operator to Sample

2.1 The Statistical Move

“99.9% of statistics is all about replacing expectations with averages.”

Suppose we want to estimate a population quantity that is an expectation under some distribution,

$$\mu = \mathbb{E}_{X \sim P_X} [g(X)].$$

We cannot compute μ directly if we do not know P_X , but if we can draw samples from P_X , we can replace the expectation with a sample average. Draw n i.i.d. samples $X_1, X_2, \dots, X_n \sim P_X$, evaluate g at each, and average:

$$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^n g(X_i).$$

By the law of large numbers $\hat{\mu}_n \rightarrow \mu$ as $n \rightarrow \infty$. The estimator updates incrementally: when a new sample X_{n+1} arrives,

$$\hat{\mu}_{n+1} = \hat{\mu}_n + \frac{1}{n+1} (g(X_{n+1}) - \hat{\mu}_n).$$

The estimate steps from $\hat{\mu}_n$ toward the new sample with a diminishing step size $1/(n+1)$. This is the running-mean form of a sample-average estimator.

This is the move we will apply to q_π . We will write $q_\pi(s, a)$ as an expectation under known distributions, identify the random variables, and replace each expectation with a sample drawn from the trajectory. The resulting estimator will have exactly the same incremental form.

2.2 The Two Forms of q_π

We have two equivalent forms of $q_\pi(s, a)$, derived in earlier reports. The definition,

$$q_\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right],$$

and the one-step Bellman expectation equation,

$$q_\pi(s, a) = (T_\pi q_\pi)(s, a) = r(s, a) + \gamma \sum_{s'} p(s' \mid s, a) \sum_{a'} \pi(a' \mid s') q_\pi(s', a').$$

The definition writes q_π as the expectation of the full return. The Bellman equation writes q_π as the expected immediate reward plus a bootstrap on q_π at the next state-action pair. We work with the Bellman form because it lets us replace expectations with samples one step at a time, instead of waiting for an entire episode.

2.3 Rewriting the Bellman Equation as a Conditional Expectation

We have the Bellman expectation equation,

$$q_\pi(s, a) = r(s, a) + \gamma \sum_{s'} p(s' \mid s, a) \sum_{a'} \pi(a' \mid s') q_\pi(s', a').$$

The right-hand side is a sum of products of probabilities and values. We want to rewrite it as a single conditional expectation over random variables drawn from the environment and the policy,

because then one trajectory step gives us one realization of the quantity inside the expectation, which is exactly what the statistical move requires.

We do this in three transformations, working from the inside out.

Transformation 1: the inner action sum. The inner sum is over actions a' at state s' , weighted by the policy $\pi(\cdot | s')$. This is the definition of an expectation: when a quantity $f(a')$ is averaged with weights $\pi(a' | s')$, the result is the expected value of f under a draw from $\pi(\cdot | s')$. Naming the random draw A_{t+1} ,

$$\sum_{a'} \pi(a' | s') q_\pi(s', a') = \mathbb{E}_{A_{t+1} \sim \pi(\cdot | s')} [q_\pi(s', A_{t+1})].$$

On the right side, s' is still a fixed argument (we have not yet started averaging over it), and A_{t+1} is the random variable. The sum has become an expectation; the value $q_\pi(s', A_{t+1})$ is now a random variable in A_{t+1} .

Transformation 2: the outer state sum. Substituting Transformation 1 back,

$$q_\pi(s, a) = r(s, a) + \gamma \sum_{s'} p(s' | s, a) \mathbb{E}_{A_{t+1} \sim \pi(\cdot | s')} [q_\pi(s', A_{t+1})].$$

The remaining outer sum is over next states s' , weighted by the environment transition $p(s' | s, a)$. By the same logic, this is the expectation of the bracketed quantity under a draw $S_{t+1} \sim p(\cdot | s, a)$:

$$\sum_{s'} p(s' | s, a) \mathbb{E}_{A_{t+1} \sim \pi(\cdot | s')} [q_\pi(s', A_{t+1})] = \mathbb{E}_{S_{t+1} \sim p(\cdot | s, a)} [\mathbb{E}_{A_{t+1} \sim \pi(\cdot | S_{t+1})} [q_\pi(S_{t+1}, A_{t+1})]].$$

Two things changed when we substituted s' with the random variable S_{t+1} . First, the argument of q_π on the inside became $q_\pi(S_{t+1}, A_{t+1})$. Second, the inner expectation's conditioning distribution became $\pi(\cdot | S_{t+1})$: the policy distribution depends on the realized next state S_{t+1} , not on a fixed s' anymore. The inner draw of A_{t+1} is made *after* the outer draw of S_{t+1} has been observed.

Transformation 3: folding the reward into the outer expectation. We now have

$$q_\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{S_{t+1} \sim p(\cdot | s, a)} [\mathbb{E}_{A_{t+1} \sim \pi(\cdot | S_{t+1})} [q_\pi(S_{t+1}, A_{t+1})]].$$

The first term is the expected immediate reward

$$r(s, a) = \mathbb{E}_{(S_{t+1}, R_{t+1}) \sim p(\cdot, \cdot | s, a)} [R_{t+1}].$$

The next-state expectation in the second term is under $S_{t+1} \sim p(\cdot | s, a)$, which is the next-state marginal of the same joint $p(\cdot, \cdot | s, a)$ that produces R_{t+1} . Since both terms are expectations under the same joint distribution, we can fold them into a single outer expectation:

$$r(s, a) + \gamma \mathbb{E}_{S_{t+1} \sim p(\cdot | s, a)} [\cdot] = \mathbb{E}_{(S_{t+1}, R_{t+1}) \sim p(\cdot, \cdot | s, a)} [R_{t+1} + \gamma [\cdot]].$$

Applying this to our equation gives the **full notational form**:

$$\boxed{q_\pi(s, a) = \mathbb{E}_{(S_{t+1}, R_{t+1}) \sim p(\cdot, \cdot | s, a)} [R_{t+1} + \gamma \mathbb{E}_{A_{t+1} \sim \pi(\cdot | S_{t+1})} [q_\pi(S_{t+1}, A_{t+1})]].}$$

The right-hand side is a single conditional expectation. What is inside the expectation is the random quantity $R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1})$, whose value is determined by the random variables (S_{t+1}, R_{t+1}) and A_{t+1} . The conditioning on $(S_t, A_t) = (s, a)$ enters through the distributions: the joint $p(\cdot, \cdot | s, a)$ depends on (s, a) , and the inner $\pi(\cdot | S_{t+1})$ depends on the realized S_{t+1} .

What we have. Fixed: $(S_t, A_t) = (s, a)$.

Random variables: R_{t+1} and S_{t+1} , drawn jointly from $p(\cdot, \cdot \mid s, a)$. A_{t+1} , drawn from $\pi(\cdot \mid S_{t+1})$, after S_{t+1} has been realized.

Sampling order: outer first, then inner. First (S_{t+1}, R_{t+1}) from the environment, then A_{t+1} from the policy at the realized S_{t+1} .

Random quantity averaged: $R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1})$.

The compact form. The same equation written under the joint policy-and-environment expectation $\mathbb{E}_\pi$, which bundles both distributions, is

$$q_\pi(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a].$$

Both forms are the same equation. The full form exposes the two distributions and the sampling order. The compact form is what we use in algorithm write-ups.

2.4 Replacing Each Expectation with One Sample

We now have $q_\pi(s, a)$ written as a conditional expectation in the form the statistical move requires:

$$q_\pi(s, a) = \mathbb{E}_{(S_{t+1}, R_{t+1}) \sim p(\cdot, \cdot \mid s, a)} [R_{t+1} + \gamma \mathbb{E}_{A_{t+1} \sim \pi(\cdot \mid S_{t+1})} [q_\pi(S_{t+1}, A_{t+1})]].$$

Match this against the template from Section 2.1, $\mu = \mathbb{E}_{X \sim P_X}[g(X)]$. The role of X is played by the tuple of random variables $(S_{t+1}, R_{t+1}, A_{t+1})$. The role of P_X is played by the sequential sampling procedure $(S_{t+1}, R_{t+1}) \sim p$ followed by $A_{t+1} \sim \pi(\cdot \mid S_{t+1})$. The role of $g(X)$ is played by the random quantity $R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1})$, and the role of μ is played by $q_\pi(s, a)$.

The statistical move says: draw one realization of X , evaluate g at it, and use that as one sample of μ . In our setting that means draw the random variables in the order specified by the nested expectations and evaluate the integrand on the realizations. The trajectory under π provides exactly these draws.

We carry out the sampling in three steps, working from outer to inner.

Draw the outer expectation. The outer expectation is over the joint $(S_{t+1}, R_{t+1}) \sim p(\cdot, \cdot \mid s, a)$. We cannot evaluate p , but the environment is itself a sampler for it. Execute action a in state s , and the environment returns one realization:

$$(S_{t+1}, R_{t+1}) \sim p(\cdot, \cdot \mid s, a).$$

This single draw replaces the outer expectation: instead of averaging over every possible next state and reward weighted by p , we have one (S_{t+1}, R_{t+1}) .

Draw the inner expectation. The inner expectation is over $A_{t+1} \sim \pi(\cdot \mid S_{t+1})$. The conditioning distribution itself depends on the realized S_{t+1} from the outer draw, which is why the inner draw must come after the outer one. The policy is a sampler we control. At the realized S_{t+1} ,

$$A_{t+1} \sim \pi(\cdot \mid S_{t+1}).$$

This single draw replaces the inner expectation: instead of averaging over every action at S_{t+1} weighted by π , we have one A_{t+1} .

Evaluate the integrand: the bootstrap. The two draws above give us realizations of all three random variables $(R_{t+1}, S_{t+1}, A_{t+1})$. We can now evaluate the integrand $R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1})$ at these realizations. The first term R_{t+1} is directly observed. The second term $q_\pi(S_{t+1}, A_{t+1})$ contains the unknown function q_π itself.

This is the bootstrap step. Since q_π is the quantity we are estimating, we cannot evaluate it. We replace it with our current best guess: the running estimate Q . Substituting,

$$\boxed{\widehat{q}_\pi(s, a) = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}).}$$

This is the SARSA target. It is one sample of the conditional expectation $q_\pi(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) \mid s, a]$, with q_π replaced by Q inside the integrand.

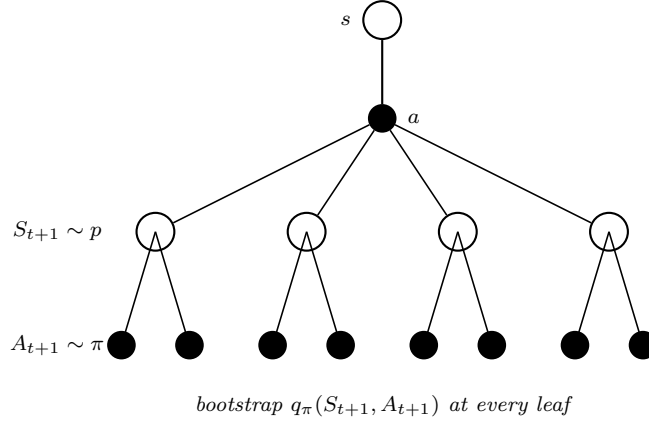
The conversion has a clean visual reading. The operator at (s, a) branches over every possible next state and, at each, branches again over every possible next action; it averages q_π over the entire tree. SARSA replaces this average with a single trajectory through the tree, one sampled next state and one sampled next action, with a bootstrap at the leaf.

The quintuple. Reading off the random variables involved in producing one sample, we see that one update uses exactly the five quantities

$$(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}).$$

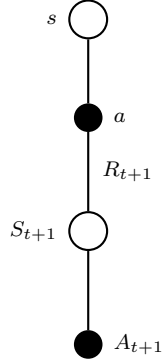
Two are fixed (the inputs (s, a)); three are sampled (the environment draw and the policy draw). This quintuple, read in order, is what gives the algorithm its name: **S**tate, **A**ction, **R**eward, next **S**tate, next **A**ction.

Operator: full expectation



$\Downarrow$ replace each $\mathbb{E}$ with one sample $\Downarrow$

SARSA sample: one trajectory



$bootstrap\ Q(S_{t+1}, A_{t+1})$

Figure 1: Backup diagram for the operator-to-sample conversion. An empty circle is a state, a filled circle is an action. *Top*: the Bellman expectation operator at (s, a) branches over every possible next state S_{t+1} drawn from $p(\cdot \mid s, a)$, and at each next state branches again over every possible next action A_{t+1} drawn from $\pi(\cdot \mid S_{t+1})$; the operator averages q_π over the entire tree below (s, a) . *Bottom*: SARSA replaces this average with a single trajectory through the tree, one sampled (S_{t+1}, R_{t+1}) , then one sampled A_{t+1} , then a bootstrap at the leaf, giving the quintuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ and the target $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$.

2.5 Running-Mean Update

We now have a procedure that produces one sample $\hat{q}_\pi(s, a) = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$ each time the agent visits (s, a) in the environment. The sample carries two sources of error. **Noise**: it reflects one realization of $(S_{t+1}, R_{t+1}, A_{t+1})$ rather than the full expectation. **Bias**: it uses Q in place of the unknown q_π inside the bootstrap. The statistical move tells us how to handle the noise:

average many samples. The bias is handled separately, by the fact that Q itself converges to q_π as the iteration proceeds (which the convergence theorem in Section 3.4 makes precise).

Recall the incremental sample-average estimator from Section 2.1. After n samples $X_1, \dots, X_n$ of a random variable, the sample average $\hat{\mu}_n$ updates on the arrival of X_{n+1} as

$$\hat{\mu}_{n+1} = \hat{\mu}_n + \frac{1}{n+1}(g(X_{n+1}) - \hat{\mu}_n).$$

Each new observation $g(X_{n+1})$ pulls the running mean toward itself with weight $1/(n+1)$.

We apply the same form to our q_π estimate. The role of $\hat{\mu}_n$ is played by $Q(s, a)$, the current estimate. The role of $g(X_{n+1})$ is played by the new sample $\hat{q}_\pi(s, a)$. Substituting,

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \underbrace{(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}))}_{\text{new sample}} - \underbrace{Q(S_t, A_t)}_{\text{current estimate}}.$$

The step size is generically written α rather than $1/(n+1)$. In the strict sample-average case $\alpha = 1/N(s, a)$, where $N(s, a)$ is the visit count. In SARSA we typically use a small constant or slowly decreasing α because the target distribution is non-stationary: the policy π is being improved at the same time as Q is being estimated, so old samples are drawn under a different policy than current samples. A constant α down-weights old samples, which helps the estimator track the changing target. We will sharpen the conditions on α when we state SARSA's convergence theorem.

This gives the SARSA(0) update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)).$$

The quantity $R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$ is called the *temporal-difference error* (TD error). It is the difference between the new sample of the right-hand side of the Bellman equation and the current estimate of the left-hand side. When Q is at the fixed point of T_π , the TD error has expectation zero. While Q is still improving, the TD error has nonzero expectation in the direction that moves Q toward q_π .

We have completed the conversion from operator to algorithm. The operator T_π on $\mathcal{Q}$ has been replaced by a single-sample estimator that is computed from one quintuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ of the trajectory, and the resulting update is the running mean of these samples. SARSA(0) is exactly this update applied at every step of every episode.

3 SARSA(0): Online GPI

3.1 From Phased GPI to Online GPI

The model-based algorithm in Section 1 runs evaluation and improvement in phases. Pick a policy π_k , iterate T_{π_k} for k steps until $Q \approx q_{\pi_k}$, then derive the next policy π_{k+1} as ε -greedy with respect to Q , then repeat. Evaluation is run to (near-)convergence before improvement is allowed to act.

SARSA(0) does not wait. Each environment step produces one running-mean update of Q , and the policy is re-derived as ε -greedy with respect to the updated Q *immediately*, before the next step. So GPI happens at the finest granularity possible: one Q update and one policy update per environment interaction. There are no separate phases.

Why this works. The Bellman expectation operator T_π has q_π as its fixed point. If π were held fixed, the running-mean updates would drive Q toward q_π in expectation, by the same argument that made the deterministic iteration $Q_{k+1} = T_\pi Q_k$ converge in Section 1. The non-stationarity introduced by changing π at every step does not break this: as π converges to greedy under the GLIE schedule (Section 3.4), T_π converges to the optimality operator T , and the running-mean updates track this converging operator. The formal statement is in Section 3.4.

3.2 On-Policy Sampling

The SARSA target derived in Section 2 was

$$\hat{q}_\pi(s, a) = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}), \quad A_{t+1} \sim \pi(\cdot \mid S_{t+1}).$$

The next action A_{t+1} used in the target is sampled from π , the policy whose action-value function we are estimating. In SARSA, this same policy is also the policy that generates the agent’s actual behaviour in the environment. The action that appears in the target is the action the agent actually takes next.

This is what *on-policy* means: the policy used to sample data is the same as the policy being evaluated. The implication is structural. At time $t + 1$, the agent observes S_{t+1} , draws $A_{t+1} \sim \pi(\cdot \mid S_{t+1})$, and uses that A_{t+1} both to bootstrap the update for (S_t, A_t) and to execute its next action in the environment. One draw, two uses. The quintuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ is read off the agent’s own trajectory; nothing is sampled hypothetically.

The contrast is with Q-learning (next report), where the target uses $\max_{a'} Q(S_{t+1}, a')$ regardless of which action the agent will actually take next. Q-learning’s target evaluates the greedy policy; its behaviour is governed by a separate exploratory policy. The two policies differ. That separation is what makes Q-learning off-policy.

3.3 The Algorithm

Algorithm 1: SARSA(0)

Input: step size $\alpha \in (0, 1]$, exploration $\varepsilon > 0$, discount γ , number of episodes N

Initialize $Q(s, a)$ arbitrarily for all s, a , with $Q(\text{terminal}, \cdot) = 0$

for each episode $e = 1, \dots, N$ **do**

 Initialize S_t at the episode start

 Choose $A_t \sim \pi(\cdot \mid S_t)$, where π is ε -greedy w.r.t. Q

while S_t is not terminal **do**

 Execute A_t ; observe R_{t+1}, S_{t+1}

 Choose $A_{t+1} \sim \pi(\cdot \mid S_{t+1})$, π ε -greedy w.r.t. Q

$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$

$S_t \leftarrow S_{t+1}, \quad A_t \leftarrow A_{t+1}$

end

end

Two structural observations on the pseudocode.

The ε -greedy policy π is implicit in Q . We do not store π as a separate table. At every action-selection step, π is computed on the fly from the current Q table via the ε -greedy rule from

Section 1.2. So when Q is updated at line 8, the policy is automatically updated at the same instant.

A_{t+1} **is the action that gets executed next.** Line 6 draws A_{t+1} from π at the new state. This is the same A_{t+1} that enters the bootstrap target on line 7 and that becomes A_t on line 9 for the next iteration. One draw, three uses: target value, next-iteration input, next environment step. This is the on-policy structure made concrete.

3.4 Convergence

SARSA(0) converges to the optimal action-value function q^* under three conditions.

Coverage. Every state-action pair is visited infinitely often. The ε -greedy rule with $\varepsilon > 0$ guarantees this in finite MDPs.

Step-size conditions. The Robbins-Monro conditions hold for the sequence (α_t) :

$$\sum_{t=0}^{\infty} \alpha_t = \infty, \quad \sum_{t=0}^{\infty} \alpha_t^2 < \infty.$$

The first ensures the running mean can still move arbitrarily far; the second ensures the variance from sampling shrinks fast enough. Typical choices: $\alpha_t = 1/t^\beta$ for $\beta \in (1/2, 1]$.

GLIE. *Greedy in the Limit with Infinite Exploration:* the policy must remain exploratory enough to cover all pairs (the first condition above) but become greedy in the limit. The standard way to satisfy this is to let $\varepsilon \rightarrow 0$ on a schedule such as $\varepsilon_k = 1/k$ across episodes. Without GLIE, SARSA converges to the action-value function of the limiting ε -soft policy, which is optimal among ε -soft policies but not globally optimal.

Key Result

SARSA(0) convergence. Under coverage, Robbins-Monro step sizes, and GLIE, the SARSA(0) iterates satisfy

$$Q_t(s, a) \xrightarrow{t \rightarrow \infty} q^*(s, a) \quad \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$$

with probability one. Proof: Singh, Jaakkola, Littman, Szepesvári (2000). The argument is stochastic approximation on the contraction T_π with π tracking Q .

The convergence statement is structural. The running-mean update inherits the contraction of T_π in expectation; the Robbins-Monro conditions control the noise; GLIE ensures the contracting operator is the one whose fixed point is q^* rather than some ε -soft compromise. We do not prove the theorem here; the full stochastic approximation machinery required (martingale convergence, ODE method) is built up in the Borkar report.